Efficient Verification of LTL via Transition Certificates

Verifying Linear Temporal Logic (LTL) specifications for discrete-time dynamical systems with continuous state spaces is critical for safety-critical applications, but the state-triplet method and closure certificate often face a trade-off between conservatism and high computational cost. To overcome these limitations, we propose transition certificates which guarantee LTL satisfaction by showing that the language of the automaton for the negated LTL specification and the language of the system are disjoint. The key insight of our approach is to decompose LTL verification into two complementary certificates: transition persistence certificates, which use ranking functions to guarantee accepting transitions take finitely often, and transition safety certificates, which leverage barrier functions to render accepting transitions unreachable. This design not only reduces the search space significantly compared to closure certificates but also mitigates conservatism inherent in state-triplet methods. We introduce automated synthesis techniques based on polynomial and neural network templates, balancing expressiveness and computational efficiency. Experimental evaluations on benchmark case studies, including the Kuramoto oscillator and a two-room temperature model, show that transition certificates achieve significant speedup over baseline methods while providing formal guarantees, highlighting their practicality for LTL verification tasks.

ACM Reference Format:
. 2018. Efficient Verification of LTL via Transition Certificates. *Proc. ACM Meas. Anal. Comput. Syst.* 37, 4, Article 111 (August 2018), 21 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

CPS have gained popularity both in industry and the research community and are represented by many varied mission critical applications [31]. The tight coupling between their computational and physical layers poses significant challenges for formal verification and safety assurance. While these models can effectively capture system behavior, their infinite state space presents challenges for verification.

Current methods for verifying temporal properties of these systems include the state-triplet method [39] and closure certificates [25]. However, these methods introduce new concerns: (i) The state-triplet method suffers from conservatism; (ii) The closure certificates have low synthesis efficiency. Developing a more efficient approach that reduces conservatism for verifying temporal properties in these systems remains a significant challenge.

Prior research has focused on formal verification of temporal properties—including safety, reachability, and reach-avoid specifications—within dynamical systems [3, 27, 41, 43, 44]. However, specifications for complex systems often require LTL [28] to capture temporal behaviors. Verifying LTL properties in control systems introduces several challenges. First, these properties describe behaviors across infinite execution trajectories [4], typically modeled via ω-automata. When employing proof certificates for verification, the state-triplet method requires unrolling all simple paths and blocking each individually. It is non-trivial to determine the number of certificates needed for synthesis and this method is conservative. Alternatively, closure certificates face the issue of enormous search spaces, as certificates are defined on $\mathcal{X} \times Q \times \mathcal{X} \times Q$. Finally, the infinite state space and nonlinear dynamics of control systems pose major obstacles for traditional verification methods.

Author's Contact Information:

ACM 2476-1249/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

In this work, we consider the formal verification of Linear Temporal Logic (LTL) properties for discrete-time dynamical systems using proof certificates. As abstraction-free methods, proof certificates have gained popularity in verification and static analysis domains [6–9]. Murali et al. [25] proposed closure certificates, proof certificates defined as real-valued functions over state pairs of dynamical systems, for persistence verification, which can be used to validate systems against LTL properties. Wongpiromsarn et al. [39] introduced the state-triplet method, which demonstrates that a system satisfies LTL properties by blocking all reachable paths. Inspired by these two methods, we propose transition safety certificates and transition persistence certificates. Their combined use enables verification of LTL specifications expressed via transition-based Nondeterministic Büchi Automata (NBA) [5]. Based on these certificates, we present an automated synthesis method using Satisfiability Modulo Theories (SMT) solvers. We show that combining these certificates reduces the conservatism of state-triplet methods while requiring smaller search spaces than closure certificates. We validate the effectiveness of our synthesis approach through two case studies.

Contributions. The main contributions of this paper are:

- We propose transition certificates, a novel notion for verifying LTL properties of discrete-time dynamical systems with continuous state spaces. Transition certificates effectively address the limitations of existing approaches: closure certificates, which suffer from enormous search spaces leading to high computational costs, and the state-triplet method, which introduces conservatism. Our approach reduces the search space to at most $\mathcal{X} \times Q \times Q$ while mitigating conservatism.
- We introduce automated synthesis methods for transition certificates based on polynomial and neural network templates. These templates provide a balance between expressiveness and computational efficiency, enabling efficient certificate generation for diverse system classes, from linear to nonlinear dynamics.
- We demonstrate the effectiveness of our certificates through case studies, including the Kuramoto oscillator and a two-room temperature model. Experimental results show that transition certificates achieve significant speedups, highlighting their practicality for LTL specifications.

The organization of this paper is as follows: Section 2 covers preliminary knowledge. Section 3 introduces our proof certificates for LTL verification and compares with other methods. Section 4 presents synthesis methods. Section 5 demonstrates the effectiveness through case studies. Section 6 concludes the paper. The proofs of the relevant lemmas are provided in the Appendix A.

Related work: Verification of temporal logic properties via proof certificates has seen significant advancements. To validate discrete-time systems against LTL specifications, Wongpiromsarn et al. [39] developed a state-triplet method that uses barrier certificates to block all simple paths from initial states to accepting states in the automaton, requiring enumeration of state triplets (q_m, q'_m, q''_m) along these paths. Murali et al. [25] introduced closure certificates for persistence verification, which establish disjunctively well-founded transition invariants [29] for dynamical systems. Additionally, substantial research has addressed the verification of specific temporal behaviors such as reachability, safety, and avoidance [6–9]. Some adopt neural networks as templates [26, 38, 42] to synthesize certificates for verifying temporal properties.

The abstraction-based approaches [10, 13, 15, 21, 24, 37, 40] lift the CPS model from continuous domain to discrete domain by computing a finite abstraction, e.g., finite transition systems and Markov decision processes (MDPs). We note that abstraction-based techniques have been used in the verification and synthesis of continuous-space dynamical systems against LTL properties such as the results in [19, 20, 23, 32]. These results rely on building a finite state abstraction and then making use of model checking and synthesis techniques [4, 35]. In general, the abstraction-based approaches are computationally demanding and suffer from the curse of dimensionality. Abstraction-free methods, such as proof certificates, can alleviate the computational burden to some extent.

In the context of our discussion, we assume that the NBA for an LTL specification ϕ has already been constructed. The translation from an LTL formula to an NBA involves an exponential complexity with respect to the size

of the formula [36]. The complexity of constructing the complement of an NBA is known to be EXPTIME [33]. Converting LTL specifications into Transition-based Generalized Büchi Automata (TGBA) typically yields smaller automata compared to Labeled Generalized Büchi Automata (LGBA) [11]. The degeneralization process [17] can convert a TGBA with multiple acceptance conditions into a Transition-based Büchi Automaton (TBA) featuring a single acceptance condition. Various tools exist for generating different automaton types from LTL formulas, such as Owl [22] and SPOT [14].

2 Preliminaries

2.1 Notation

We use $\mathbb{N}$ and $\mathbb{R}$ to represent the set of natural numbers and the set of real numbers, respectively. We define:

$$\mathbb{N}_{\sim i} = \{n \in \mathbb{N} \mid n \sim i\}, \quad \text{where } i \in \mathbb{N}, {\sim} \in \{\geq, \leq, >, <\},$$
$$\mathbb{R}_{\sim i} = \{r \in \mathbb{R} \mid r \sim i\}, \quad \text{where } i \in \mathbb{R}, {\sim} \in \{\geq, \leq, >, <\}.$$

Given a set S and a set S_1, we define an indicator function:

$$I_S(x) = \begin{cases} 1, & \text{if } x \in S \\ 0, & \text{if } x \notin S. \end{cases}$$

$|S|$ denotes the cardinality of S and $S \backslash S_1$ denotes $\{x \in S \mid x \notin S_1\}$.

Given a sequence $s = \langle s_0, s_1, \dots \rangle$, where $s_i \in S$ for $i \in \mathbb{N}$:

- $s[i]$ denotes s_i;
- $s[i : j]$ denotes $\langle s_i, \dots, s_j \rangle$ for $j \geq i$;
- $s[i : \infty]$ denotes $\langle s_i, s_{i+1}, \dots \rangle$.

We say s visits S if there exists $i \in \mathbb{N}$ such that $s_i \in S$. $Inf(s)$ denotes the set of elements that appear in s infinitely often. S^ω denotes the set of infinite sequences over S, i.e., $S^\omega = \{s \mid s = \langle s_0, s_1, \dots \rangle \text{ and } s_i \in S \text{ for } i \in \mathbb{N}\}$. A function $f : S \to \mathbb{R}$ is bounded if and only if there exist $a, b \in \mathbb{R}$ such that $a \leq f(x) \leq b$ for all $x \in S$.

2.2 Discrete-time Dynamical System

A discrete-time dynamical system $\mathfrak{S}$ is a tuple $(\mathcal{X}, \mathcal{X}_0, f)$, where $\mathcal{X} \subseteq \mathbb{R}^n$ represents the state set, $\mathcal{X}_0 \subseteq \mathcal{X}$ represents a set of initial states, and $f : \mathcal{X} \to \mathcal{X}$ represents the transition function. The evolution of the system state is described as follows:

$$x_{t+1} = f(x_t).$$

An infinite sequence of states $\langle x_0, x_1, x_2, \dots \rangle$ is called a *trajectory* of the system if $x_0 \in \mathcal{X}_0$ and $x_{i+1} = f(x_i)$ for $i \in \mathbb{N}$. We use $\mathfrak{T}(\mathfrak{S})$ to represent all *trajectories* on $\mathfrak{S}$. The *predecessor* of x_{i+1} is x_i for $i \in \mathbb{N}$. In a given *trajectory*, x_{i+1} has only one *predecessor*. We use the notation x_i^{pre} to denote the *predecessor* of x_i in a *trajectory*.

2.3 Specification

While this paper focuses on the verification of LTL properties for discrete-time dynamical systems, we also cover foundational concepts such as safety, persistence, and Nondeterministic Büchi Automata (NBA), which are essential for establishing our framework.

Safety: A system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ is *safe* with respect to unsafe states $\mathcal{X}_u \subseteq \mathcal{X}$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we have $x_i \notin \mathcal{X}_u$ for all $i \in \mathbb{N}$.

Persistence: A system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ is *persistent* with respect to states $\mathcal{X}_{VF} \subseteq \mathcal{X}$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, elements of $\mathcal{X}_{VF}$ appear finitely often in τ.

Linear Temporal Logic (LTL) [28]: LTL formulae are defined over a set of atomic propositions AP. A labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ maps system states to subsets of AP. Given a *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we denote $\mathfrak{L}(\tau) = \langle \mathfrak{L}(x_0), \mathfrak{L}(x_1), \dots \rangle$ as the corresponding *word*.

The syntax of LTL is:

$$\phi := \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi U \phi,$$

where $a \in AP$ is an atomic proposition. Temporal operators finally (F) and always (G) are derived from next (X) and until (U).

Given a *word* $w = \mathfrak{L}(\tau)$ and an LTL formula ϕ, the semantics is defined inductively:

- $w \models \top$ always holds,
- $w \models a$ iff $a \in w[0]$,
- $w \models \phi_1 \wedge \phi_2$ iff $w \models \phi_1$ and $w \models \phi_2$,
- $w \models \neg\phi_1$ iff $w \not\models \phi_1$,
- $w \models X\phi_1$ iff $w[1:\infty] \models \phi_1$,
- $w \models \phi_1 U \phi_2$ iff there exists $j \in \mathbb{N}$ such that $w[j:\infty] \models \phi_2$ and for all $i \in \mathbb{N}_{<j}$, $w[i:\infty] \models \phi_1$.

We say $\mathfrak{S} \models_{\mathfrak{L}} \phi$ if for any $\tau \in \mathfrak{T}(\mathfrak{S})$, we have $\mathfrak{L}(\tau) \models \phi$.

Nondeterministic Büchi Automaton (NBA) [18]: An NBA is a tuple $(\Sigma, Q, q_0, \delta, Acc)$, where $\Sigma = 2^{AP}$ is the alphabet, Q is a finite set of states, $q_0 \in Q$ is the initial state, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, and $Acc \subseteq \delta$ is the set of *accepting transitions*. Given a word $w = \langle w_0, w_1, \dots \rangle \in \Sigma^\omega$, a **run** on w is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$ and $(r_i, w_i, r_{i+1}) \in \delta$ for all $i \in \mathbb{N}$. The *transition-based accepting condition* is that an NBA accepts w if there exists a run where at least one accepting transition occurs infinitely often, i.e., $Inf(r) \cap Acc \neq \emptyset$.

For an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, we define:

$$Acc^{pair} = \{(p, q) \mid \exists \sigma \in \Sigma. (p, \sigma, q) \in Acc\}, \tag{1}$$

$$Acc^{start} = \{p \mid \exists \sigma \in \Sigma, q \in Q. (p, \sigma, q) \in Acc\}, \tag{2}$$

$$Acc^{k,l} = \{(q_k, \sigma, q_l) \mid (q_k, \sigma, q_l) \in Acc\}. \tag{3}$$

We denote $\mathfrak{L}(\mathfrak{S}) = \{\mathfrak{L}(\tau) \mid \tau \in \mathfrak{T}(\mathfrak{S})\}$ as the *language* of $\mathfrak{S}$, and $\mathfrak{L}(\mathcal{A})$ as the language accepted by $\mathcal{A}$. To verify $\mathfrak{S} \models_{\mathfrak{L}} \phi$, we construct an NBA $\mathcal{A}$ for $\neg\phi$ and show $\mathfrak{L}(\mathfrak{S}) \cap \mathfrak{L}(\mathcal{A}) = \emptyset$.

2.4 Certificates

Barrier certificates and closure certificates are used to LTL properties [28], respectively. We provide their definitions and related conclusions from prior work.

Definition 2.1 (barrier certificate [30]). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_u \subseteq \mathcal{X}$, a bounded function $B(x) : \mathcal{X} \to \mathbb{R}$ is a *barrier certificate* with respect to $\mathcal{X}_u$ such that $x_0 \in \mathcal{X}_0, x_u \in \mathcal{X}_u, x \in \mathcal{X}$ and $x' = f(x)$. We have:

$$B(x_0) \geq 0, \tag{4}$$

$$B(x_u) < 0, \tag{5}$$

$$B(x) \geq 0 \Rightarrow B(x') \geq 0. \tag{6}$$

Theorem 2.2 (Safety [30]). *The existence of a barrier certificate $B(x)$ implies that the system $\mathfrak{S}$ is safe with respect to $\mathcal{X}_u$.*

The state-triplet approach[39] uses barrier certificates to verify LTL properties. Given an LTL specification ϕ, it proves that accepting states are unreachable in the NBA for $\neg\phi$ by finding barrier certificates to ensure that runs on $\mathfrak{L}(\mathfrak{S})$ will never visit accepting states, verifying $\mathfrak{S} \models_{\mathfrak{L}} \phi$. The state-triplet method need to find barrier

certificates for each simple path, a certificate corresponding to a specific state triplet to individually block a simple path. The state-triplet method introduces conservatism: it cannot be applied for systems that satisfy the LTL specification because they cannot be proven safe under this restrictive constraints.

Definition 2.3 (closure certificate [25]). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ represent $\neg\phi$. A bounded function $T : \mathcal{X} \times Q \times \mathcal{X} \times Q \to \mathbb{R}$ is a closure certificate for ϕ if there exists a value $\epsilon \in \mathbb{R}_{>0}$ such that for all states $x, y, y' \in \mathcal{X}$, $x' = f(x)$, states $q_i, q_j \in Q$, and $q_i' \in \delta(q_i, \mathfrak{L}(x))$, we have:

$$T((x, q_i), (x', q_i')) \geq 0, \tag{7}$$

$$T((x', q_i'), (y, q_j)) \geq 0 \Rightarrow T((x, q_i), (y, q_j)) \geq 0. \tag{8}$$

and for all states $x_0 \in \mathcal{X}_0$, and $\ell, \ell' \in Acc$, we have:

$$\begin{aligned} &T((x_0, q_0), (y, \ell)) \geq 0 \wedge T((y, \ell), (y', \ell')) \geq 0 \\ &\Rightarrow T((x_0, q_0), (y', \ell')) \leq T((x_0, q_0), (y, \ell)) - \epsilon. \end{aligned} \tag{9}$$

Theorem 2.4 (LTL [25]). *Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an LTL formula ϕ. Let an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ represent $\neg\phi$. The existence of a closure certificate satisfying conditions (7)-(9) implies that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

Closure certificates are defined by characterizing an over-approximation of the transition closure along with a decrease condition, ensuring that transitions in certain regions are visited only finitely often, and are used for LTL verification. Closure certificates (2.3) are defined on $\mathcal{X} \times Q \times \mathcal{X} \times Q$, creating high-dimensional search spaces with heavy computational burdens.

2.5 Problem Statement

In this paper, we investigate the following LTL verification problem.

Problem. Given a discrete-time dynamical system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP, a labeling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$, and an LTL formula ϕ, verify that $\mathfrak{S} \models_{\mathfrak{L}} \phi$ holds.

Closure certificates require searching a large space, making synthesis slow, and the state-triplet method can be overly conservative. In this paper, we propose a new approach to address both issues.

3 Transition Certificates

We propose a new type of proof certificates, called transition certificates, to verify that a system $\mathfrak{S}$ satisfies an LTL formula ϕ. The core idea of transition certificates is to reduce the verification problem to analyzing the behavior of accepting transitions in the NBA $\mathcal{A}$ for $\neg\phi$. By the definition of the transition-based acceptance condition, a word is accepted by $\mathcal{A}$ if and only if accepting transitions occur infinitely often along one of its runs. Therefore, to prove $\mathfrak{S} \models_{\mathfrak{L}} \phi$, it suffices to show that for all trajectories of $\mathfrak{S}$, every accepting transition in $\mathcal{A}$ occurs at most finitely many times. We introduce two complementary types of transition certificates: *transition safety certificates* and *transition persistence certificates*. While each type is theoretically sufficient for LTL verification on its own, their combined use is crucial for practical effectiveness, as it significantly reduces conservatism. The two certificates address the problem from fundamentally different angles:

- A *transition safety certificate* proves that the system's trajectories can never reach Acc^{start}, thereby guaranteeing that related accepting transitions **never occur**.
- A *transition persistence certificate* allows the system to reach Acc^{start} but proves that related accepting transitions can only be taken **finitely often**.

3.1 Transition Safety Certificate

To verify that accepting transitions never occur, we show that the starting states of these transitions, denoted by Acc^{start}, are unreachable in the product of the system and the automaton.

Definition 3.1 (transition safety certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$. Construct the product system $\mathfrak{S} \times \mathcal{A} = (\mathcal{Y}, \mathcal{Y}_0, F)$, where $\mathcal{Y} = \{(x, q) \mid x \in \mathcal{X}, q \in Q\}$, $\mathcal{Y}_0 = \{(x_0, q_0) \mid x_0 \in \mathcal{X}_0\}$, and $F(x, q) = \{(x', q') \mid x' = f(x) \wedge (q, \mathfrak{L}(x), q') \in \delta\}$. For $q_k \in Acc^{start}$, define the unsafe set $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$. A bounded function $B_k(x, q) : \mathcal{X} \times Q \to \mathbb{R}$ is a *transition safety certificate* with respect to $q_k \in Acc^{start}$ if for all $(x, q) \in \mathcal{Y}$, $(x_0, q_0) \in \mathcal{Y}_0$, $(x', q') \in F(x, q)$, we have:

$$B_k(x_0, q_0) \geq 0, \tag{10}$$

$$B_k(x, q_k) < 0, \tag{11}$$

$$B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0. \tag{12}$$

Intuitively,

- condition (10) guarantees that the certificate value is non-negative at the initial state of the product system;
- condition (11) ensures that the certificate becomes negative precisely on the unsafe set $\mathcal{Y}_u^k$ associated with q_k, thereby marking states where accepting transitions could originate;
- condition (12) implies that once the certificate value is non-negative at a state, it remains non-negative along any trajectory of the product system, thus preserving the safety invariant.

Collectively, these conditions ensure that the certificate B_k guarantees q_k is unreachable, preventing all accepting transitions from q_k. The following lemma formalizes this guarantee. Its proof is provided in Appendix A.

Lemma 3.2. *If a transition safety certificate B_k with respect to $q_k \in Acc^{start}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ never visit state q_k, thereby preventing all accepting transitions starting from q_k.*

The next theorem follows directly from applying the above lemma to every state in Acc^{start}. If a transition safety certificate can be found for each state in Acc^{start}, then no accepting transition can be taken, ensuring the LTL specification holds.

Theorem 3.3. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ for $\neg\phi$. If for any $q_i \in Acc^{start}$, a transition safety certificate B_i can be found, then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

Proof. By leveraging Lemma 3.2, all start states of accepting transitions are unreachable. Consequently, all accepting transitions never occur, ensuring that the system's language is not accepted and thereby guaranteeing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. □

3.2 Transition Persistence Certificate

In contrast, the *transition persistence certificate* offers a less conservative approach. Instead of completely preventing the system from reaching states where accepting transitions may happen, this certificate proves that even if such states are reached, the corresponding accepting transitions can occur only a finite number of times. This is achieved by requiring a decrease in a certificate value each time an accepting transition is taken.

Definition 3.4 (transition persistence certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$. For $(k, l) \in Acc^{pair}$, define the indicator function $I_{k,l}(x) = 1$ if $(q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}$, and 0 otherwise. A bounded function $B_{k,l}(x) : \mathcal{X} \to \mathbb{R}$ is a *transition persistence certificate*

with respect to $Acc^{k,l}$ if there exists $\epsilon > 0$ such that for all $x_0 \in \mathcal{X}_0$, $x \in \mathcal{X}$, $x' = f(x)$, we have:

$$B_{k,l}(x_0) \geq 0, \tag{13}$$

$$B_{k,l}(x) \geq 0 \Rightarrow B_{k,l}(x') \geq 0, \tag{14}$$

$$B_{k,l}(x) \geq B_{k,l}(x') + I_{k,l}(x)\epsilon. \tag{15}$$

Intuitively,

- condition (13) guarantees that the certificate $B_{k,l}$ is non-negative at the initial state of the system;
- condition (14) ensures that $B_{k,l}$ remains non-negative along any trajectory of the system;
- condition (15) implies that $B_{k,l}$ decreases by at least ϵ whenever the state's label matches an accepting transition from q_k to q_l, and remains non-increasing otherwise.

Since accepting transitions from q_k to q_l can only occur when $(q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}$, the number of such transitions is bounded by $B_{k,l}(x_0)/\epsilon$, ensuring finite occurrence. This yields the following lemma. Its proof is provided in the Appendix A.

Lemma 3.5. *If a transition persistence certificate $B_{k,l}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

The next theorem is obtained by applying the lemma to every accepting transition pair.

Theorem 3.6. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ for $\neg\phi$. if for any $(k, l) \in Acc^{pair}$, a corresponding transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

Proof. By leveraging Lemma 3.5, all accepting transitions occur only finitely often, which ensures that the system's language is not accepted, thereby guaranteeing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. □

While both Theorem 3.3 and Theorem 3.6 can verify LTL properties, the two types of transition certificates yield better effects when used in combination. In some scenarios, requiring the system to completely avoid certain states might be too strict, making transition safety certificates impossible to find. However, the system might still satisfy the property if the accepting transitions occur only finitely often, a condition that can be certified by a transition persistence certificate. Conversely, consider verifying $\mathbf{G}\neg a$ using the NBA for $\mathbf{F}a$, where $Acc^{0,0} = \{(q_0, \emptyset, q_0), (q_0, \{a\}, q_0)\}$. For the persistence certificate $B_{0,0}$, condition (15) requires $B_{0,0}(x) \geq B_{0,0}(f(x)) + \epsilon$ for any x where an accepting transition is enabled. This would demand an infinite descent in the bounded certificate value, which is impossible, preventing the transition persistence certificate from being found. Yet, a transition safety certificate can successfully verify the property by proving that state q_0 (the start of the accepting transitions) is unreachable in the product system. Thus, providing both certificates covers a wider range of system behaviors, enhancing the method's applicability. We now present the main theorem for LTL verification. For each accepting transition start state in the NBA, we use either a transition safety certificate or transition persistence certificates to prove accepting transitions from that state occur finitely often.

In summary, the combination of transition safety certificates and transition persistence certificates offers a less conservative verification framework. It maintains a smaller search space compared to closure certificates while providing two distinct mechanisms to ensure that accepting transitions occur only finitely often, ultimately proving that the system satisfies its LTL specification.

4 Certificate Synthesis

This section develops automated synthesis techniques for transition certificates. We leverage the Counterexample-Guided Inductive Synthesis (CEGIS) framework [34], a well-established paradigm in barrier certificate synthesis,

to iteratively refine candidate solutions. By alternating between candidate generation and formal verification, this approach ensures that the discovered certificates are valid and proves that the system meets the specified LTL properties. Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ representing $\neg\phi$, we adopt a uniform notation where B_k represents the transition safety certificate template for states $q_k \in Acc^{\text{start}}$, and $B_{k,l}$ represents the transition persistence certificate template for $Acc^{k,l}$. To find transition certificates, we present CEGIS frameworks for polynomial and neural network templates, respectively. The synthesis proceeds in two alternating phases: *candidate generation* and *verification*.

4.1 Polynomial CEGIS

4.1.1 Transition Safety Certificates. We define a parametric polynomial template:

$$B_k(x, q; \mathbf{c}) = \sum_{i=1}^{m} c_i \, p_i(x, q), \tag{16}$$

where $x \in \mathcal{X}$, $q \in Q$, $\{p_1, \ldots, p_m\}$ is a user-defined basis of monomials over $\mathcal{X} \times Q$ with degree at most d, and $\mathbf{c} = (c_1, \ldots, c_m) \in \mathbb{R}^m$ are unknown coefficients. Our goal is to find $\mathbf{c}$ such that $B_k(\cdot, \cdot; \mathbf{c})$ satisfies conditions (10), (11), and (12).

Candidate Generation. Based on certificate conditions, we encode the constraint for all points of the sampled dataset. For transition safety certificates, the constraint is:

$$\bigwedge_{(x,q)\in D_0} B_k(x, q; \mathbf{c}) \geq 0 \tag{17}$$

$$\wedge \bigwedge_{(x,q)\in D_u} B_k(x, q; \mathbf{c}) < 0 \tag{18}$$

$$\wedge \bigwedge_{\substack{(x,q)\in D_{\mathcal{X}} \\ (q,\mathfrak{L}(x),q')\in\delta}} \big[B_k(x, q; \mathbf{c}) \geq 0 \Rightarrow B_k(f(x), q'; \mathbf{c}) \geq 0\big], \tag{19}$$

where $D_0, D_u, D_{\mathcal{X}}$ are the datasets sampled from $\mathcal{X}_0 \times \{q_0\}$, $\mathcal{X} \times \{q_k\}$, and $\mathcal{X} \times Q$, respectively. The coefficient synthesis involves logical implications (disjunctive constraints) between linear inequalities. We employed Z3 [12] as an SMT solver to directly handle the logical structure. If the constraint is satisfiable with a solution $\hat{\mathbf{c}}$, define the candidate certificate as $\hat{B}_k = B_k(\cdot, \cdot; \hat{\mathbf{c}})$.

Verification. To verify $\hat{B}_k$ is valid, we formulate the negation of conditions and verify they are unsatisfiable. For transition safety certificates, we check:

$$x \in \mathcal{X}_0 \wedge \hat{B}_k(x, q_0) < 0, \tag{20}$$

$$x \in \mathcal{X} \wedge \hat{B}_k(x, q_k) \geq 0, \tag{21}$$

$$x \in \mathcal{X} \wedge (q, \mathfrak{L}(x), q') \in \delta \wedge \hat{B}_k(x, q) \geq 0 \wedge \hat{B}_k(f(x), q') < 0. \tag{22}$$

We choose dReal [16] as our SMT verifier to check these constraints. If the constraints are satisfiable (i.e., counterexamples exist), we add them to D_0, D_u, or $D_{\mathcal{X}}$ accordingly. The synthesis process then iterates until no counterexamples are found, yielding a valid transition safety certificate.

4.1.2 Transition Persistence Certificates. We define a parametric polynomial template:

$$B_{k,l}(x; \mathbf{c}) = \sum_{i=1}^{m} c_i \, p_i(x), \tag{23}$$

where $x \in \mathcal{X}$, $\{p_1, \ldots, p_m\}$ are monomials over $\mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^m$ are unknown coefficients. We seek $\mathbf{c}$ and $\epsilon > 0$ satisfying conditions (13), (14), and (15).

Candidate Generation. Based on certificate conditions, we encode the constraint for all points of the sampled dataset. For transition persistence certificates, the constraint is:

$$\bigwedge_{x \in D_0} B_{k,l}(x; \mathbf{c}) \geq 0 \tag{24}$$

$$\wedge \bigwedge_{x \in D_{\mathcal{X}}} \left[B_{k,l}(x; \mathbf{c}) \geq 0 \Rightarrow B_{k,l}(f(x); \mathbf{c}) \geq 0 \right] \tag{25}$$

$$\wedge \bigwedge_{\substack{x \in D_{\mathcal{X}} \\ (q_k, \mathfrak{L}(x), q_l) \notin Acc^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) \tag{26}$$

$$\wedge \bigwedge_{\substack{x \in D_{\mathcal{X}} \\ (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) + \epsilon, \tag{27}$$

where $D_0, D_{\mathcal{X}}$ are the datasets sampled from $\mathcal{X}_0$ and $\mathcal{X}$, respectively. We solve the coefficient synthesis problem using the Z3 solver [12]. If satisfiable with solution $(\hat{\mathbf{c}}, \hat{\epsilon})$, define the candidate transition persistence certificate as $\hat{B}_{k,l} = B_{k,l}(\cdot; \hat{\mathbf{c}})$.

Verification. To verify $\hat{B}_{k,l}$ is valid, we encode constraints of the negation of conditions and verify they are unsatisfiable. For transition persistence certificates, we check:

$$x \in \mathcal{X}_0 \wedge \hat{B}_{k,l}(x) < 0, \tag{28}$$

$$x \in \mathcal{X} \wedge \hat{B}_{k,l}(x) \geq 0 \wedge \hat{B}_{k,l}(f(x)) < 0, \tag{29}$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)) + \hat{\epsilon}, \tag{30}$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{L}(x), q_l) \notin Acc^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)). \tag{31}$$

We use dReal [16] to check these constraints. If the constraints are satisfiable, we add them to D_0 or $D_{\mathcal{X}}$ accordingly. The synthesis process then iterates until no counterexamples are found, yielding a valid transition persistence certificate.

4.2 Neural Network CEGIS

Since neural networks have the capability to approximate arbitrary functions, using them as templates for certificates offers advantages over polynomials [42]. Assume that $x \in \mathbb{R}^n$ represents the n-dimensional input vector of the neural network, $L-1$ represents the number of hidden layers, and y is the scalar output of the neural network with respect to the input x. Thus, the feedforward neural network can be expressed with the following structure:

$$\begin{cases} z_l = \mathbf{W}_l x_{l-1} + \mathbf{b}_l, & 0 < l \leqslant L \\ x_l = \sigma(z_l), & 0 < l \leqslant L \\ y = x_L \end{cases} \tag{5}$$

where,

- z_l denotes the output vector of the linear operation in the l-th layer;
- $\mathbf{W}_l$ and $\mathbf{b}_l$ denote the weight matrix and bias vector for the l-th layer of the neural network, respectively;

- σ denotes the activation function. Common activation functions include the Tanh function, Sigmoid function, ReLU function.

Candidate Generation. Based on certificate conditions, a loss function is designed for the training dataset. For transition safety certificates, the loss function is:

$$\begin{aligned}\mathcal{L}_B = &\sum_{(x,q)\in D_0} \mathrm{ReLU}(-B_k(x,q;\theta)) \\ &+ \sum_{(x,q)\in D_u} \mathrm{ReLU}(B_k(x,q;\theta)) \\ &+ \sum_{\substack{(x,q)\in D_{\mathcal{X}} \\ (q,\mathfrak{L}(x),q')\in\delta}} \mathrm{ReLU}(B_k(x,q;\theta)) \cdot \mathrm{ReLU}(-B_k(f(x),q';\theta)),\end{aligned} \tag{32}$$

where $D_0, D_u, D_{\mathcal{X}}$ are datasets sampled from $\mathcal{X}_0 \times \{q_0\}$, $\mathcal{X} \times \{q_k\}$, and $\mathcal{X} \times Q$, respectively. The terms in the loss function offer intuitive measures of the extent to which the neural network violates the certificate conditions.

For transition persistence certificates, the loss function is:

$$\mathcal{L}_B = \sum_{\substack{x\in D_{\mathcal{X}} \\ (q_k,\mathfrak{L}(x),q_l)\in Acc^{k,l}}} \mathrm{ReLU}(B_{k,l}(f(x);\theta) + \epsilon - B_{k,l}(x;\theta)) \tag{33}$$

$$+ \sum_{\substack{x\in D_{\mathcal{X}} \\ (q_k,\mathfrak{L}(x),q_l)\notin Acc^{k,l}}} \mathrm{ReLU}(B_{k,l}(f(x);\theta) - B_{k,l}(x;\theta)), \tag{34}$$

where $D_{\mathcal{X}} \subseteq \mathcal{X}$ is the dataset sampled from $\mathcal{X}$.

To generate a neural certificate candidate, we employ gradient descent techniques to minimize the loss function. When the loss decreases to 0, it indicates that the learned neural network can be a candidate certificate. In addition, the activation functions differ: the transition safety certificate employs the linear activation function, while the transition persistence certificate uses the squared activation function. This design eliminates the need to verify non-negativity conditions during verification. We define the candidate transition safety certificate as $\hat{B}_k = B_k(\cdot,\cdot;\hat{\theta})$ and the candidate transition persistence certificate to be $\hat{B}_{k,l} = B_{k,l}(\cdot;\hat{\theta})$.

Verification. To verify the candidate neural certificate is valid, we formulate the negation of conditions and use dReal [16] to verify they are unsatisfiable. For transition safety certificates, we check constraints (20), (21), and (22). For transition persistence certificates, we verify constraints (30) and (31). If counterexamples are found, they are added to the training set accordingly, and the synthesis process iterates until no counterexamples remain.

We choose dReal [16] as our SMT verifier, which ensures the correctness of its unsat decisions. Therefore, when dReal returns unsat for the given formula, it confirms that no solution exists within the specified precision, and the candidate certificate is valid. Otherwise, it means that dReal has found a counterexample that violates the certificate conditions. These counterexamples are added to the sample sets and the candidate generation phase is repeated. The process iterates until all verification queries are unsatisfiable (within the δ-completeness threshold of dReal), yielding a valid certificate.

5 Experimental Evaluation

We evaluate the proposed transition certificate synthesis methods on two benchmark systems to demonstrate: (1) their effectiveness in formal LTL verification; and (2) their computational efficiency compared to (neural) closure certificate methods. For the first benchmark, the Kuramoto Oscillator model (covering both 1D and 2D scenarios), we verify the system against specified LTL properties. For the second benchmark, the two-room temperature

model, we demonstrate the versatility of our approach by synthesizing transition certificates to satisfy complex LTL requirements and highlight their complementary advantages in handling diverse temporal specifications. All experiments are conducted on Ubuntu 22.04.3 LTS (Intel i7-9750H, 8 GB RAM). During candidate generation, Z3 addresses the constrained optimization for polynomial templates, whereas neural network templates are trained using the Adam optimizer (learning rate 0.001, 1000 iterations) to minimize the loss function. For formal verification, we employ dReal (precision 0.0001) to handle the transcendental sin functions in the Kuramoto Oscillator model, while Z3 is utilized for the temperature model.

5.1 Kuramoto Oscillator

5.1.1 One-Dimensional System. The first benchmark is a Kuramoto oscillator system, formally defined as $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ with dynamics adapted from [1]. The system operates within the state space $\mathcal{X} = [0, 2\pi]$, starting from an initial region $\mathcal{X}_0 = [\frac{4\pi}{9}, \frac{5\pi}{9}]$. To evaluate LTL verification, we specify an unsafe region $\mathcal{X}_u = [\frac{7\pi}{9}, \frac{8\pi}{9}]$ within the temporal properties. The discrete-time transition function f is given by:

$$f(x) = x + t_s\Omega + t_sK\sin(-x) - 0.532x^2 + 1.69, \tag{35}$$

where x represents the oscillator phase. The system parameters are set as follows: sampling time $t_s = 0.1$, natural frequency $\Omega = 0.01$, and coupling strength $K = 0.0006$.

We consider the LTL specification $\phi = \mathbf{G}\neg a$ over the atomic proposition set $AP = \{a\}$, with the labeling function defined as $\mathfrak{L}(x) = \{a\}$ for $x \in \mathcal{X}_u$ and $\mathfrak{L}(x) = \emptyset$ otherwise. The negated specification, $\neg\phi = \mathbf{F}a$, is characterized by the NBA illustrated in Figure 1. This automaton incorporates accepting transitions $(q_0, \emptyset, q_0)$ and (q_0, a, q_0) originating from the initial state $q_0 \in Acc^{\text{start}}$.

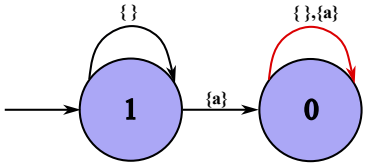

Fig. 1. NBA representing the negated specification $\mathbf{F}a$. Accepting transitions (highlighted in red) are depicted as self-loops at state q_0.

Based on the definition of transition safety certificates, we implement a polynomial-based CEGIS framework to synthesize certificates that guarantee the satisfaction of the specified LTL property. The polynomial template is structured as follows:

$$B_0(x, q; \mathbf{c}) = \sum_{i=0}^{4} c_i x^i + \sum_{j=1}^{3} c_{4+j} q^j + c_8 xq.$$

The synthesis process begins by collecting 30 samples from region D_0, 600 from D_u, and 1200 from $D_{\mathcal{X}}$. The candidate generation phase is formulated as a constraint satisfaction problem and solved using the Z3 solver. Given that the system dynamics involve the transcendental sin function, which lies beyond the decision procedures of Z3, we employ the dReal solver during the verification phase to identify counterexamples. The complete CEGIS loop converges in 3.6 seconds, successfully yielding a valid transition certificate:

$$B_0(x, q) = -1 + 2q^2 - \frac{7}{16}xq.$$

In addition to polynomial templates, we synthesize a neural transition certificate $B_0(x, q; \theta)$ using the CEGIS framework. The certificate is parameterized by a feedforward neural network with a 2-10-1 architecture, featuring

two input nodes for the pair (x, q), a hidden layer of 10 neurons with ReLU activation, and a single linear output. The training process is initialized with a dataset of 250 samples (40 from D_0, 70 from D_u, and 140 from $D_{\mathcal{X}}$). During each iteration, the dReal solver is employed to identify any points violating the certificate conditions. These identified counterexamples are then incorporated into the training set to progressively refine the candidate certificate. The complete CEGIS loop converges in 3.2 seconds. The valid certificates obtained during the process are provided in the appendix.

5.1.2 Two-Dimensional System. The benchmark is further extended to a two-dimensional scenario with state space $\mathcal{X} = [0, \frac{8\pi}{9}]^2$ and initial region $\mathcal{X}_0 = [0, \frac{\pi}{9}]^2$. In this case, the unsafe region $\mathcal{X}_u$ is defined as the boundary corridors of the state space:

$$\mathcal{X}_u = \left([0, \tfrac{8\pi}{9}] \times [\tfrac{5\pi}{6}, \tfrac{8\pi}{9}]\right) \cup \left([\tfrac{5\pi}{6}, \tfrac{8\pi}{9}] \times [0, \tfrac{8\pi}{9}]\right). \tag{36}$$

The coupled system dynamics f are governed by:

$$f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + t_s \begin{bmatrix} \Omega + 1.69 \\ \Omega + 1.69 \end{bmatrix} + K t_s \begin{bmatrix} \sin(x_2 - x_1) \\ \sin(x_1 - x_2) \end{bmatrix} - 0.532 t_s \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}. \tag{37}$$

To maintain consistency with the 1D analysis, we define the labeling function such that $\mathfrak{L}(x_1, x_2) = \{a\}$ if $(x_1, x_2) \in \mathcal{X}_u$ and $\emptyset$ otherwise. The verification is performed against the same LTL specification $\phi = \mathbf{G}\neg a$ using the NBA illustrated in Figure 1.

For the two-dimensional system, we first employ a polynomial template $B_0(x_1, x_2, q; \mathbf{c})$. The synthesis process utilizes a dataset of 30 samples from D_0, 729 from D_u, and 1,458 from $D_{\mathcal{X}}$. Within 7.3 seconds, the CEGIS loop yields a valid transition certificate:

$$B_0(x_1, x_2, q) = -1 + \tfrac{1}{8}x_1x_2 + \tfrac{3}{2}q^2 - \tfrac{3}{32}qx_1^2 - \tfrac{153}{512}x_2q. \tag{38}$$

Furthermore, we implement a neural network template with a 3-15-1 architecture, featuring three input nodes for the tuple (x_1, x_2, q) and a hidden layer of 15 neurons with ReLU activation. To demonstrate the efficiency of our framework, we initialize the training with a set of points: 40 from D_0, 90 from D_u, and 180 from $D_{\mathcal{X}}$. The CEGIS loop converges in 623.4 seconds, successfully producing a valid neural certificate in appendix.

5.2 Two-Room Temperature

The second benchmark focuses on the formal verification of an interconnected temperature control system, denoted by $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, with its dynamical model adapted from [2]. The system operates within a state space $\mathcal{X} = [20, 34]^2$, originating from the initial configuration $\mathcal{X}_0 = [21, 24]^2$. The underlying LTL specification pertains to the target region $\mathcal{X}_{VF} = [20, 26]^2$. Specifically, the evolution of the system state is governed by the following discrete-time dynamics f:

$$f(x_1, x_2) = \begin{bmatrix} 1 - 2\alpha - \theta - \mu u(x_1) & \alpha \\ \alpha & 1 - 2\alpha - \theta - \mu u(x_2) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \mu T_h \begin{bmatrix} u(x_1) \\ u(x_2) \end{bmatrix} + \theta \begin{bmatrix} T_e \\ T_e \end{bmatrix}, \tag{39}$$

where the constant parameters are assigned as $\alpha = 0.004$, $\theta = 0.01$, $\mu = 0.15$, $T_h = 40$, and $T_e = 0$. The state-dependent control term is defined as $u(x_i) = 0.59 - 0.011x_i$ for $i \in \{1, 2\}$.

Consider the set of atomic propositions $AP = \{a, b\}$, where the labeling function is defined as $\mathfrak{L}(x_1, x_2) = \{a\}$ for $(x_1, x_2) \in \mathcal{X}_0$, and $b \in \mathfrak{L}(x_1, x_2)$ for $(x_1, x_2) \in \mathcal{X}_{VF}$. The system is required to satisfy the LTL specification $\varphi = a \Rightarrow \mathbf{FG}\neg b$. The corresponding NBA, which captures the negation of the specification (i.e., $\neg\varphi = a \wedge \mathbf{GF}b$), is illustrated in Figure 2. To demonstrate the complementary roles of transition safety certificates and transition persistence certificates, we synthesize both types of certificates for this benchmark.

Consider the transition safety certificate template and transition persistence certificate template(seen in Appendix B). We simultaneously synthesize them. For the transition safety certificate, we collect 16 samples for

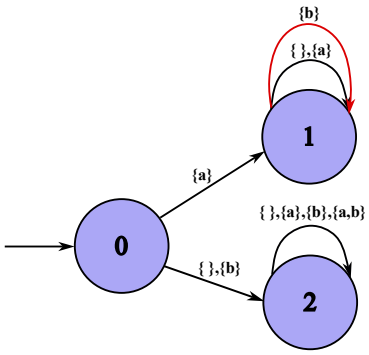

Fig. 2. NBA for $a \wedge \mathbf{GF}b$ (negation of $a \Rightarrow \mathbf{FG}\neg b$). Accepting transitions (red) are $(q_1, \{b\}, q_1)$ and $(q_1, \{a, b\}, q_1)$.

D_0, 196 for D_u, 588 for $D_{\mathcal{X}}$. For the transition persistence certificate, we collect 16 samples for D_0 and 196 for $D_{\mathcal{X}}$. We obtain a valid transition persistence certificate

$$B_{1,1}(x_1, x_2) = 27I_{[20,26]^2}(x_1, x_2) - x_1 I_{[20,26]^2}(x_1, x_2)$$

and $\epsilon = 0.25$ in 8.8 seconds.

We synthesize the transition safety certificate $B_1(x_1, x_2, q)$ for q_1 and the transition persistence certificate $B_{1,1}(x_1, x_2)$ for $Acc^{1,1}$. We synthesize $B_1(x_1, x_2, q)$ using the CEGIS with neural network template of architecture 3-10-1 (3 inputs for (x_1, x_2, q), 10 hidden neurons with ReLU activation, 1 output). We synthesize $B_{1,1}(x_1, x_2)$ using the CEGIS with neural network template of architecture 2-3-1 (2 inputs for (x_1, x_2), 3 hidden neurons with squared activation, 1 output). For the transition persistence certificate, we collect 100 samples for $D_{\mathcal{X}}$. For the transition safety certificate, we collect 4 samples for D_0, 9 for D_u, 18 for $D_{\mathcal{X}}$. The entire CEGIS loop terminates in 1228.3 seconds and the valid certificates obtained are provided in the appendix..

5.3 Comparison

We primarily compare our approach with closure certificates [25] and neural closure certificates [26], as they represent recent developments in abstraction-free LTL verification. While the state-triplet method [39] is a foundational approach, it is omitted from our quantitative comparison for two reasons: (1) its inherent conservatism often fails to find valid certificates for complex LTL specifications, particularly those involving persistence properties; and (2) closure certificates [25] were specifically developed to provide more general verification conditions than the state-triplet method. The core idea of closure certificates lies in characterizing an over-approximation of the transition closure, so closure certificates for LTL verification are defined over $\mathcal{X} \times \mathcal{X} \times Q \times Q$. In contrast, our method employs transition safety certificates to directly filter out unreachable automaton states or uses transition persistence certificates to prove that the accepting transitions occur finitely often, so the search space of transition certificates is at most $\mathcal{X} \times Q \times Q$. This allows us to reduce the search space from $\mathcal{X} \times \mathcal{X} \times Q \times Q$ to $\mathcal{X} \times Q \times Q$ and improves synthesis efficiency. For neural templates, we adopt the CEGIS framework, whereas the neural closure certificate approach relies on sampling and posteriori verification: after sampling the state space with a given density ϵ and safety margin η, the method trains until the loss converges to zero and then performs a posteriori verification to determine whether the synthesized certificate is valid.

We conduct two groups of comparisons: (1) polynomial-template CEGIS versus closure certificates, and (2) neural-template CEGIS versus neural closure certificates. The detailed experimental configurations are provided in Appendix B. Table 1 and Table 2 summarize the total computation time, where "Timeout" indicates that the synthesis exceeded one hour. As shown in Table 1, when suitable templates are selected, our polynomial template

Table 1. Synthesis time comparison (seconds)

Benchmark	Transition Certificates	Closure Certificate
Kuramoto 1D	3.6	4.2
Kuramoto 2D	7.3	Timeout
Two-Room Temperature	8.8	Timeout

Table 2. Synthesis time comparison (seconds)

Benchmark	Transition Certificates	Neural Closure Certificate
Kuramoto 1D	3.2	Timeout
Kuramoto 2D	234.1	Timeout
Two-Room Temperature	1228.3	Timeout

based CEGIS method completes synthesis within seconds. In contrast, the closure certificate method synthesizes a certificate quickly only in the 1D case under given template; for the other cases using the templates reported in the original paper, the method times out, which is consistent with the results reported in the original work where synthesis typically requires over an hour. Similarly, as shown in Table 2, the neural network template CEGIS requires approximately 10–20 minutes, which is slower than the polynomial-template version but remains considerably faster than the neural closure certificate baseline. The neural closure certificate approach, due to its high sampling density, exhibits substantially longer training times, a result that aligns with the original paper's reported verification times of roughly 4–5 hours. These comparisons confirm that our synthesis methods achieve a significant speedup over both closure certificates and neural closure certificates, demonstrating the efficiency gains obtained from reducing search space complexity.

6 Conclusion

In this paper, we have proposed transition certificates as an efficient framework for verifying Linear Temporal Logic (LTL) properties of discrete-time dynamical systems with continuous state spaces. Our approach leverages complementary certificates—transition safety and persistence certificates—to reduce conservatism compared to state-triplet methods and minimize search spaces relative to closure certificates, achieving significant speedups in LTL verification. We have developed automated synthesis methods based on polynomial and neural network templates, demonstrating practicality through case studies like the Kuramoto oscillator and two-room temperature model. Nonetheless, our method faces a few shortcomings that motivate future work. First, the synthesis process, particularly for neural network templates, can be computationally demanding for high-dimensional systems with highly nonlinear dynamics, due to the bottleneck in SMT solving. Second, the framework relies on the construction of NBA for LTL specifications, which involves exponential complexity and may limit scalability to complex formulas. Additionally, the current work focuses on discrete-time systems, leaving continuous-time systems as an open challenge. Future work will explore several directions. We plan to investigate the application of transition certificates to controller synthesis, enabling safe control policies with formal guarantees. Another direction is to adapt the notions for continuous-time systems, extending the verification framework to broader classes of dynamics. We also aim to integrate learning techniques, such as inferring specifications from trajectory data or automating template selection, to enhance adaptability and reduce manual effort. The transition certificates framework has potential applications beyond LTL verification, including the analysis of cyber-physical systems, AI-based controllers, and other domains requiring temporal logic guarantees. For instance, it could be used to ensure safety in autonomous systems or verify protocols in distributed computing. By providing a less

conservative and efficient verification method, our work contributes to reliable system design, with implications for safety-critical areas like robotics and smart infrastructure. We will further explore these directions in future research.

References

[1] Mahathi Anand, Abolfazl Lavaei, and Majid Zamani. 2022. From small-gain theory to compositional construction of barrier certificates for large-scale stochastic systems. *IEEE Trans. Automat. Control* 67, 10 (2022), 5638–5645.

[2] Mahathi Anand, Abolfazl Lavaei, and Majid Zamani. 2024. Compositional synthesis of control barrier certificates for networks of stochastic systems against ω-regular specifications. *Nonlinear Analysis: Hybrid Systems* 51 (2024), 101427.

[3] Matin Ansaripour, Krishnendu Chatterjee, Thomas A Henzinger, Mathias Lechner, and Đorđe Žikelić. 2023. Learning provably stabilizing neural controllers for discrete-time stochastic systems. In *International Symposium on Automated Technology for Verification and Analysis*. Springer, 357–379.

[4] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of model checking*. MIT press.

[5] J Richard Büchi. 1966. Symposium on decision problems: On a decision method in restricted second order arithmetic. In *Studies in Logic and the Foundations of Mathematics*. Vol. 44. Elsevier, 1–11.

[6] Aleksandar Chakarov and Sriram Sankaranarayanan. 2013. Probabilistic program analysis with martingales. In *Computer Aided Verification: 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings 25*. Springer, 511–526.

[7] Krishnendu Chatterjee, Hongfei Fu, Petr Novotnỳ, and Rouzbeh Hasheminezhad. 2016. Algorithmic analysis of qualitative and quantitative termination problems for affine probabilistic programs. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 327–342.

[8] Krishnendu Chatterjee, Amir Goharshady, Ehsan Goharshady, Mehrdad Karrabi, and Đorđe Žikelić. 2024. Sound and complete witnesses for template-based verification of LTL properties on polynomial programs. In *International Symposium on Formal Methods*. Springer, 600–619.

[9] Krishnendu Chatterjee, Ehsan Kafshdar Goharshady, Petr Novotnỳ, and Đorđe Žikelić. 2024. Equivalence and similarity refutation for probabilistic programs. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 2098–2122.

[10] Samuel Coogan, Ebru Aydin Gol, Murat Arcak, and Calin Belta. 2015. Traffic network control from temporal logic specifications. *IEEE Transactions on Control of Network Systems* 3, 2 (2015), 162–172.

[11] Jean-Michel Couvreur. 1999. On-the-fly verification of linear temporal logic. In *International Symposium on Formal Methods*. Springer, 253–271.

[12] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.

[13] Xuchu Ding, Stephen L Smith, Calin Belta, and Daniela Rus. 2014. Optimal control of Markov decision processes with linear temporal logic constraints. *IEEE Trans. Automat. Control* 59, 5 (2014), 1244–1257.

[14] Alexandre Duret-Lutz, Alexandre Lewkowicz, Amaury Fauchille, Thibaud Michaud, Étienne Renault, and Laurent Xu. 2016. Spot 2.0 — A Framework for LTL and omega-Automata Manipulation. In *Automated Technology for Verification and Analysis*, Cyrille Artho, Axel Legay, and Doron Peled (Eds.). Springer International Publishing, Cham, 122–129.

[15] Georgios E Fainekos, Antoine Girard, Hadas Kress-Gazit, and George J Pappas. 2009. Temporal logic motion planning for dynamic robots. *Automatica* 45, 2 (2009), 343–352.

[16] Sicun Gao, Soonho Kong, and Edmund M Clarke. 2013. dReal: An SMT solver for nonlinear theories over the reals. In *International conference on automated deduction*. Springer, 208–214.

[17] Dimitra Giannakopoulou and Flavio Lerda. 2002. From states to transitions: Improving translation of LTL formulae to Büchi automata. In *International Conference on Formal Techniques for Networked and Distributed Systems*. Springer, 308–326.

[18] Ernst M Hahn, Mateo Perez, Sven Schewe, Fabio Somenzi, Ashutosh Trivedi, and Dominik Wojtczak. 2020. Reward shaping for reinforcement learning with omega-regular objectives. *arXiv preprint arXiv:2001.05977* (2020).

[19] Thomas A Henzinger, Pei-Hsin Ho, and Howard Wong-Toi. 1997. HyTech: A model checker for hybrid systems. In *International conference on computer aided verification*. Springer, 460–463.

[20] Mahmoud Khaled and Majid Zamani. 2021. OmegaThreads: symbolic controller design for ω-regular objectives. In *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control*. 1–7.

[21] Hadas Kress-Gazit, Georgios E Fainekos, and George J Pappas. 2009. Temporal-logic-based reactive mission and motion planning. *IEEE transactions on robotics* 25, 6 (2009), 1370–1381.

[22] Jan Křetínský, Tobias Meggendorfer, and Salomon Sickert. 2018. Owl: A Library for omega-Words, Automata, and LTL. In *Automated Technology for Verification and Analysis*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer International Publishing, Cham, 543–550.

[23] Morteza Lahijanian, Sean B Andersson, and Calin Belta. 2011. Temporal logic motion planning and control with probabilistic satisfaction guarantees. *IEEE Transactions on Robotics* 28, 2 (2011), 396–409.

[24] Kevin Leahy, Eric Cristofalo, Cristian-Ioan Vasile, Austin Jones, Eduardo Montijano, Mac Schwager, and Calin Belta. 2019. Control in belief space with temporal logic specifications using vision-based localization. *The International Journal of Robotics Research* 38, 6 (2019), 702–722.
[25] Vishnu Murali, Ashutosh Trivedi, and Majid Zamani. 2024. Closure certificates. In *Proceedings of the 27th ACM International Conference on Hybrid Systems: Computation and Control*. 1–11.
[26] Alireza Nadali, Vishnu Murali, Ashutosh Trivedi, and Majid Zamani. 2024. Neural closure certificates. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21446–21453.
[27] Grigory Neustroev, Mirco Giacobbe, and Anna Lukina. 2025. Neural continuous-time supermartingale certificates. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27538–27546.
[28] Amir Pnueli. 1977. The temporal logic of programs. In *18th annual symposium on foundations of computer science (sfcs 1977)*. ieee, 46–57.
[29] Andreas Podelski and Andrey Rybalchenko. 2004. Transition invariants. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science, 2004.* IEEE, 32–41.
[30] Stephen Prajna and Ali Jadbabaie. 2004. Safety verification of hybrid systems using barrier certificates. In *International workshop on hybrid systems: Computation and control*. Springer, 477–492.
[31] Ragunathan Rajkumar, Insup Lee, Lui Sha, and John Stankovic. 2010. Cyber-physical systems: the next computing revolution. In *Proceedings of the 47th design automation conference*. 731–736.
[32] Matthias Rungger and Majid Zamani. 2016. SCOTS: A tool for the synthesis of symbolic controllers. In *Proceedings of the 19th international conference on hybrid systems: Computation and control*. 99–104.
[33] S. Safra. 1988. On the complexity of omega-automata. In *[Proceedings 1988] 29th Annual Symposium on Foundations of Computer Science*. 319–327. doi:10.1109/SFCS.1988.21948
[34] Armando Solar-Lezama. 2008. *Program synthesis by sketching*. University of California, Berkeley.
[35] Paulo Tabuada. 2009. *Verification and control of hybrid systems: a symbolic approach*. Springer Science & Business Media.
[36] Moshe Y Vardi. 2005. An automata-theoretic approach to linear temporal logic. *Logics for concurrency: structure versus automata* (2005), 238–266.
[37] Cristian-Ioan Vasile, Kevin Leahy, Eric Cristofalo, Austin Jones, Mac Schwager, and Calin Belta. 2016. Control in belief space with temporal logic specifications. In *2016 IEEE 55th Conference on Decision and Control (CDC)*. IEEE, 7419–7424.
[38] Peixin Wang, Jianhao Bai, Dapeng Zhi, Min Zhang, and Luke Ong. [n. d.]. Verifying Omega-regular Properties of Neural Network-Controlled Systems via Proof Certificates. In *ICLR 2025 Workshop: VerifAI: AI Verification in the Wild*.
[39] Tichakorn Wongpiromsarn, Ufuk Topcu, and Andrew Lamperski. 2016. Automata theory meets barrier certificates: Temporal logic verification of nonlinear systems. *IEEE Trans. Automat. Control* 61, 11 (2016), 3344–3355.
[40] Tichakorn Wongpiromsarn, Ufuk Topcu, and Richard M Murray. 2009. Receding horizon temporal logic planning for dynamical systems. In *Proceedings of the 48h IEEE Conference on Decision and Control (CDC) held jointly with 2009 28th Chinese Control Conference*. IEEE, 5997–6004.
[41] Bai Xue, Renjue Li, Naijun Zhan, and Martin Fränzle. 2021. Reach-avoid analysis for stochastic discrete-time systems. In *2021 American Control Conference (ACC)*. IEEE, 4879–4885.
[42] Hengjun Zhao, Xia Zeng, Taolue Chen, and Zhiming Liu. 2020. Synthesizing barrier certificates using neural networks. In *Proceedings of the 23rd international conference on hybrid systems: Computation and control*. 1–11.
[43] Đorđe Žikelić, Mathias Lechner, Thomas A Henzinger, and Krishnendu Chatterjee. 2023. Learning control policies for stochastic systems with reach-avoid guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 11926–11935.
[44] Đorđe Žikelić, Mathias Lechner, Abhinav Verma, Krishnendu Chatterjee, and Thomas Henzinger. 2023. Compositional policy learning in stochastic control systems with formal guarantees. *Advances in Neural Information Processing Systems* 36 (2023), 47849–47873.

A Proof

Theorem A.1. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, if a transition safety certificate $B_k(x, q)$ with respect to $q_k \in Acc^{start}$ can be found, then it can be guaranteed that all runs of all* words *in $\mathfrak{L}(\mathfrak{S})$ cannot visit the state q_k.*

Proof. By Definition 3.1, we have $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$ as the unsafe set in the product system $\mathfrak{S} \times \mathcal{A}$. We verify that B_k satisfies the barrier certificate conditions (4, 5, 6) with respect to $\mathcal{Y}_u$:

- Condition (10) states $B_k(x_0, q_0) \geq 0$ for all $(x_0, q_0) \in \mathcal{Y}_0$, which corresponds to condition (4).
- Condition (11) states $B_k(x, q_k) < 0$ for all $(x, q_k) \in \mathcal{Y}_u$, which corresponds to condition (5).

- Condition (12) states $B_k(x,q) \geq 0 \Rightarrow B_k(x',q') \geq 0$ for all $(x',q') \in F(x,q)$, which corresponds to condition (6).

By Theorem (2.2), any trajectory of the product system $\mathfrak{S} \times \mathcal{A}$ cannot visit $\mathcal{Y}_u$.

Now we establish the connection to automaton runs. For any word $w = \mathfrak{L}(\tau) \in \mathfrak{L}(\mathfrak{S})$ where $\tau = \langle x_0, x_1, \dots \rangle$ is a trajectory of $\mathfrak{S}$, the corresponding run r of w on $\mathcal{A}$ is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$. The sequence of product states $\langle (x_0, r_0), (x_1, r_1), \dots \rangle$ forms a trajectory in $\mathfrak{S} \times \mathcal{A}$. Since this trajectory cannot visit $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$, we have $r_i \neq q_k$ for all i. Therefore, runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit state q_k. □

Theorem A.2. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, and $(k,l) \in Acc^{pair}$, if a transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that all runs of the words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

Proof. We define $\mathcal{X}_{k,l} = \{x \mid (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}\}$ as the set of states where accepting transitions from q_k to q_l can occur.

Consider any trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$ and its corresponding word $w = \mathfrak{L}(\tau)$. Let r be a run of w on $\mathcal{A}$ starting from q_0. An accepting transition from q_k to q_l can occur at step i only if $(q_k, \mathfrak{L}(x_i), q_l) \in Acc^{k,l}$, which is equivalent to $x_i \in \mathcal{X}_{k,l}$. Therefore, the number of accepting transitions from q_k to q_l is bounded by the number of times the trajectory visits $\mathcal{X}_{k,l}$.

By Definition 3.4, we have:

- $B_{k,l}(x_0) \geq 0$ by condition (13).
- $B_{k,l}(x_i) \geq 0$ for all i by condition (14), since $B_{k,l}$ remains non-negative along the trajectory.
- $B_{k,l}(x_i) \geq B_{k,l}(x_{i+1}) + I_{k,l}(x_i)\epsilon$ by condition (15), where $I_{k,l}(x_i) = 1$ if $x_i \in \mathcal{X}_{k,l}$ and 0 otherwise.

Suppose the trajectory visits $\mathcal{X}_{k,l}$ at indices $i_1, i_2, \dots, i_m$ where m is the number of times accepting transitions occur. At each visit, $B_{k,l}$ decreases by at least ϵ:

$$B_{k,l}(x_0) \geq B_{k,l}(x_{i_1}) + \epsilon \geq B_{k,l}(x_{i_2}) + 2\epsilon \geq \cdots \geq B_{k,l}(x_{i_m}) + m\epsilon. \tag{40}$$

Since $B_{k,l}(x_{i_m}) \geq 0$ by condition (14), we have $B_{k,l}(x_0) \geq m\epsilon$, which gives $m \leq \frac{B_{k,l}(x_0)}{\epsilon}$. Therefore, the number of accepting transitions from q_k to q_l is bounded, ensuring they occur only finitely often. □

B Experiment

B.1 Template for Two-Dimensional Kuramoto Oscillator System

The transition safety certificate template for the two-dimensional Kuramoto Oscillator system is given by:

$$\begin{aligned} B_0(x_1, x_2, q) = {} & c_0 + c_1 \cdot x_1 + c_2 \cdot x_2^2 + c_3 \cdot x_1 x_2 + c_4 \cdot x_1^3 \\ & + c_5 \cdot q + c_6 \cdot q^2 + c_7 \cdot q x_1^2 + c_8 \cdot x_2 q \end{aligned}$$

B.2 Template for Two-Room Temperature System

For the *transition safety certificate*:

$$\begin{aligned} B_1(x_1, x_2, q) = {} & c_0 + c_1 \cdot I_{X_0}(x_1, x_2) + c_2 \cdot I_{X_{VF}}(x_1, x_2) \\ & + c_3 \cdot x_1 I_{X_0}(x_1, x_2) + c_4 \cdot x_2 I_{X_0}(x_1, x_2) \\ & + c_5 \cdot x_1 I_{X_{VF}}(x_1, x_2) + c_6 \cdot x_2 I_{X_{VF}}(x_1, x_2) \\ & + c_7 \cdot \max(x_1, x_2) + c_8 \cdot x_1^2 + c_9 \cdot x_2^2 + c_{10} \cdot q + c_{11} \cdot q^2. \end{aligned}$$

The template for the *transition persistence certificate* is as follows:

$$\begin{aligned} B_{1,1}(x_1, x_2) = c_0 &+ c_1 \cdot I_{X_0}(x_1, x_2) + c_2 \cdot I_{X_{VF}}(x_1, x_2) \\ &+ c_3 \cdot x_1 I_{X_0}(x_1, x_2)) + c_4 \cdot x_2 I_{X_0}(x_1, x_2) \\ &+ c_5 \cdot x_1 I_{X_{VF}}(x_1, x_2) + c_6 \cdot x_2 I_{X_{VF}}(x_1, x_2) \\ &+ c_7 \cdot \max(x_1, x_2) + c_8 \cdot x_1^2 + c_9 \cdot x_2^2. \end{aligned}$$

B.3 Neural Network Certificate Synthesis Results

This section provides the neural network certificates synthesized in the three case studies.

B.3.1 Kuramoto Oscillator: One-Dimensional System. **Valid transition safety certificate network parameters:**

- **Network architecture:** $2 \longrightarrow 10 \longrightarrow 1$ (ReLU activation network)
- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{10\times 2}$:

$$W^{(1)} = \begin{bmatrix} 0.36732605 & -0.29466873 \\ 0.44511154 & -0.10065039 \\ -0.2463612 & 0.13090049 \\ 0.5710599 & 0.18457386 \\ -0.16401595 & 0.800196 \\ 0.16680917 & -0.54196787 \\ 0.03139566 & -0.581802 \\ -0.2696909 & -0.6400563 \\ 0.6605623 & 0.6877757 \\ -0.12304431 & 0.4023705 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^{10}$:

$$b^{(1)} = \begin{bmatrix} 0.3557 & -0.1530 & -0.2855 & -0.3364 & 0.1495 \\ 0.1568 & 0.2077 & 0.4496 & -0.1997 & -0.8233 \end{bmatrix}^\top$$

- **Layer 2 weights** $W^{(2)} \in \mathbb{R}^{1\times 10}$:

$$W^{(2)} = \begin{bmatrix} 0.12380187 & 0.04241462 & -0.10408211 & -0.1692231 & 0.32869837 \\ -0.3943727 & -0.09415235 & -0.1155376 & 0.12839615 & -0.03449793 \end{bmatrix}$$

- **Layer 2 bias** $b^{(2)} \in \mathbb{R}$:

$$b^{(2)} = -0.12770885$$

- **Analytical expression:**

$$B(x, q) = W^{(2)} \cdot \mathrm{ReLU}(W^{(1)} \cdot [x, q]^\top + b^{(1)}) + b^{(2)}$$

B.3.2 Kuramoto Oscillator: Two-Dimensional System. **Valid transition safety certificate network parameters:**

- **Network architecture:** $3 \longrightarrow 15 \longrightarrow 1$ (ReLU activation network)

- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{15\times 3}$:

$$W^{(1)} = \begin{bmatrix} 0.16947323 & -0.21574873 & 0.13164099 \\ -0.08310825 & -0.01274627 & -0.04113036 \\ -0.06397208 & -0.54293144 & -0.02944631 \\ -0.31079152 & 0.06346325 & -0.08720846 \\ -0.08576902 & 0.09323277 & 0.05105818 \\ 0.3797034 & -0.6118731 & 0.02912938 \\ -0.12490943 & -0.05650124 & -0.19199097 \\ -1.1089448 & 0.63800216 & -0.06427222 \\ -0.17578183 & 0.6446198 & -0.03190435 \\ 0.14327501 & 0.08219448 & -0.22752807 \\ -0.08726892 & 0.63300693 & 0.099369 \\ -0.01736208 & 0.02708694 & 0.01039273 \\ 0.77797484 & -0.3705437 & -0.0628436 \\ 0.49392858 & 0.17566358 & 0.6906795 \\ -0.65604967 & -0.7597173 & 0.65568864 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^{15}$:

$$b^{(1)} = \begin{bmatrix} -0.3938 & -0.0656 & -0.0435 & -0.3020 & -0.1906 & -0.0847 & -0.0903 \\ -0.3402 & -1.0316 & -0.5757 & -1.0281 & -0.0510 & -0.7655 & -0.6736 \\ 0.2616 \end{bmatrix}^\top$$

- **Layer 2 weights** $W^{(2)} \in \mathbb{R}^{1\times 15}$:

$$W^{(2)} = \begin{bmatrix} 0.00767429 & -0.01955141 & -0.21718296 & 0.17923115 & 0.00531028 \\ -0.35746995 & 0.00876291 & -1.0855321 & -1.4935373 & -0.13878155 \\ -0.44775316 & -0.00077066 & -0.72719783 & 0.2176193 & 0.21799223 \end{bmatrix}$$

- **Layer 2 bias** $b^{(2)} \in \mathbb{R}$:

$$b^{(2)} = -0.14020069$$

- **Analytical expression:**

$$B(x_1, x_2, q) = W^{(2)} \cdot \mathrm{ReLU}(W^{(1)} \cdot [x_1, x_2, q]^\top + b^{(1)}) + b^{(2)}$$

B.3.3 Two-Room Temperature System. **Valid transition persistence certificate network parameters:**

- **Network architecture:** $2 \to 3 \to 1$
- **Decrease constant** ϵ**:** 0.01 (fixed)
- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{3\times 2}$:

$$W^{(1)} = \begin{bmatrix} 0.09350396 & -0.06689734 \\ 0.3247081 & -0.3753044 \\ 0.8092528 & -0.7572314 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^3$:

$$b^{(1)} = \begin{bmatrix} -0.76164836 & 1.549208 & -1.5467515 \end{bmatrix}^\top$$

B.4 Comparative Method Setting

B.4.1 Closure Certificates Method. The synthesis process uses Z3 to find candidate certificates and dReal to find counterexamples, with a dReal precision of 0.0001.

One-Dimensional Kuramoto Oscillator System: The NBA is defined as $(\Sigma, Q, q_0, \delta, \text{Acc})$ to represent Fa.

$$\begin{aligned}
\text{Acc} &= \{0\} \\
Q &= \{1, 0\} \\
q_0 &= 1 \\
\Sigma &= \{\emptyset, \{a\}\} \\
\delta &= \{(1, \emptyset, 1), (1, \{a\}, 0), (0, \emptyset, 0), (0, \{a\}, 0)\} \\
a &\text{ is defined as } x \in X_{\text{u}}
\end{aligned}$$

The certificate templates are:

$$\begin{aligned}
T_{0,0} &= c_0 + c_1 x + c_2 y \\
T_{0,1} &= c_3 + c_4 x + c_5 y \\
T_{1,0} &= c_6 + c_7 x + c_8 y \\
T_{1,1} &= c_9 + c_{10} x + c_{11} y
\end{aligned}$$

The final synthesized coefficients are shown in Table 3.

Table 3. Coefficients for the synthesized closure certificates (One-Dimensional System)

Template	c_0	c_1	c_2
$T_{0,0}(x, y)$	0.000000	0.000000	0.000000
$T_{0,1}(x, y)$	-0.500000	0.000000	0.000000
$T_{1,0}(x, y)$	-4.310550	1.764117	0.000000
$T_{1,1}(x, y)$	0.000000	0.000000	0.000000

Two-Dimensional Kuramoto Oscillator System: The certificate template is defined for $i, j \in \{0, 1\}$ as:

$$\begin{aligned}
T_{i,j}((x_1, x_2), (y_1, y_2)) = {} & c_{0,i,j} + c_{1,i,j} y_1 I_{X_0}(x_1, x_2) + c_{2,i,j} y_2 I_{X_0}(x_1, x_2) \\
& + c_{3,i,j} y_1 I_{X_u}(x_1, x_2) + c_{4,i,j} y_2 I_{X_u}(x_1, x_2) \\
& + c_{5,i,j} y_1 + c_{6,i,j} y_2
\end{aligned}$$

Two-Room Temperature System: The NBA for $a_0 \wedge Fa_1$ is $(\Sigma, Q, q_0, \delta, \text{Acc})$.

$$
\begin{aligned}
Q &= \{0, 1, 2, 3\} \\
\Sigma &= \{\emptyset, \{a_0\}, \{a_1\}, \{a_0, a_1\}\} \\
q_0 &= 0 \\
\text{Acc} &= \{2\} \\
\delta &= \{(q_0, \{a_0\}, q_1), (q_0, \{a_1\}, q_1), (q_0, \{a_0, a_1\}, q_2), (q_0, \emptyset, q_3), \\
&\quad (q_0, \{a_1\}, q_3), (q_1, \emptyset, q_1), (q_1, \{a_0\}, q_1), (q_1, \{a_1\}, q_2), \\
&\quad (q_1, \{a_0, a_1\}, q_2), (q_2, \emptyset, q_2), (q_2, \{a_0\}, q_2), (q_2, \{a_1\}, q_2), \\
&\quad (q_2, \{a_0, a_1\}, q_2), (q_3, \emptyset, q_3), (q_3, \{a_0\}, q_3), (q_3, \{a_1\}, q_3), \\
&\quad (q_3, \{a_0, a_1\}, q_3)\} \\
a_0 &\text{ is defined as } x \in X_0 \\
a_1 &\text{ is defined as } x \in X_{VF}
\end{aligned}
$$

The certificate template for this system is defined for $i, j \in \{0, 1, 2, 3\}$ as:

$$
\begin{aligned}
T_{i,j}((x_1, x_2), (y_1, y_2)) = {} & c_{0,i,j} + c_{1,i,j}x_1 + c_{2,i,j}x_2 + c_{3,i,j}y_1 + c_{4,i,j}y_2 \\
& + c_{5,i,j}\max(x_1, x_2) + c_{6,i,j}\max(y_1, y_2) \\
& + c_{7,i,j}x_1^2 + c_{8,i,j}x_2^2 + c_{9,i,j}y_1^2 + c_{10,i,j}y_2^2
\end{aligned}
$$

B.4.2 Neural Closure Certificates Method. The configuration for the neural closure certificates is summarized in Table 4. A sampling density ξ of 0.01 and a safety margin η of 0.01 are used for all systems. The Lipschitz constant is estimated using the spectral norm-based algorithm.

Table 4. Neural network architecture and parameters for Neural Closure Certificates

System	Network Architecture	Activation	Lipschitz Estimation
1D Kuramoto Oscillator	(4-10-1)	ReLU	Spectral Norm
2D Kuramoto Oscillator	(6-10-1)	ReLU	Spectral Norm
Two-Room Temperature	(6-10-1)	ReLU	Spectral Norm